

Sri Lanka Institute of Information Technology

Human Computer Interaction (IT3060)

Lab Exercise 05



Details

Campus: Malabe Campus

Name: Jeewantha R M C

Year: 3rd year 2nd Semester

Group: WE 1.1

Date: 17/09/2026

Contents

Activity 1: Compare Flutter, React Native, Kotlin and Swift.....	3
1.1 Strengths, trade-offs and scope	3
1.2 Ten-criterion comparison (qualitative).....	3
1.3 Recommended frontend and justification	3
Activity 2: Prepare Test Tasks and Success Criteria	4
2.1 Backend frameworks.....	4
2.2 Database choices	4
2.3 Identity and authorization options.....	5
2.4 Combined technology choice	5
2.5 Combined technology choice	5
Activity 3: Technology comparison and weighted decision matrix.....	6
3.1 Method, weights and scoring conventions.....	6
3.2 Worked calculation and sensitivity check	7
3.3 Decision rationale and case-study reconciliation.....	7
Activity 4: High-level architecture.....	7
4.1 Proposed architecture diagram.....	7
4.2 Component responsibilities	8
4.3 Critical data flows.....	8
Flow A - Personalized workout planning	8
Flow B - Private social sharing.....	8
Flow C - Nutrition logging	8
4.4 Cross-cutting quality and integration.....	9
4.5 Architecture Decision Record (ADR-001)	9
References and source provenance.....	9

Activity 1: Compare Flutter, React Native, Kotlin and Swift

1.1 Strengths, trade-offs and scope

Option	Strengths for FitFlow	Limitations / risks
Flutter (Dart)	Shared mobile/web UI, consistent rendering, strong animation toolkit; Flutter web supports browser deployment [R3].	Dart onboarding; Flutter-rendered web interface may be less ideal than semantic web components for some web-heavy/SEO needs; native ML bridges must be tested.
React Native + Expo (TypeScript)	Shared Android/iOS UI and business logic, familiar React skills, Expo web support; New Architecture is current default [R1,R2].	Some ML/camera/native modules need platform-specific code and compatibility checks; web UX and accessibility still require separate review.
Kotlin Multiplatform + Compose	Share business logic and optionally UI; native interoperability; Compose UI stable on Android/iOS, web target beta [R4].	Web UI maturity and staffing are project risks; iOS build toolchain; larger coordination if separate native interfaces.
Swift + SwiftUI	Strong Apple-platform tooling and deep iOS/watch integrations [R5].	Does not directly ship Android/web; separate Android/web stacks and teams raise duplication for FitFlow.

1.2 Ten-criterion comparison (qualitative)

Criterion	Flutter	React Native	Kotlin MP	Swift / SwiftUI
Development speed	Fast after Dart ramp-up	Fast for TypeScript/React team	Medium; platform configuration	Fast iOS; slow total 3-platform
Code reusability	High shared UI/logic	High shared UI/logic; web variations	High logic; UI optional	Low outside Apple
Performance	Good compiled rendering; profile	Good with New Architecture; profile	Near-native shared logic; UI varies	Strong Apple native
Ecosystem	Broad packages	Large JS/React + Expo ecosystem	Strong Kotlin, varied web tooling	Strong Apple ecosystem
Learning curve	Dart + widgets	React/TS; native bridges	Kotlin plus iOS/web concepts	Swift + second platform stack
Web compatibility	Built-in Flutter web	Expo web + React DOM [R1]	Compose web beta [R4]	Needs separate web app
AI/ML integration	Native plugin / platform bridge	On-device module or bridge; Python API	Native framework interop	Apple Core ML; others separate
Real-time features	WebSocket libraries	WebSocket/Socket.IO clients	Platform networking options	Native clients plus others
Maintenance cost	One main codebase	One main app + minor platform code	Shared logic, possible 3 UIs	Separate Android/web maintenance
Security	Depends on implementation	Depends on implementation	Depends on implementation	Depends on implementation

1.3 Recommended frontend and justification

Proposed choice: React Native with Expo and TypeScript for iOS and Android, plus Expo web/React for the user-facing web application. This preserves the case study's React Native direction while enabling reusable navigation, domain models, validation, and selected UI components across three targets [S1, R1]. The New Architecture supports the current React

Native ecosystem, but actual FitFlow screens must be profiled on representative devices rather than assuming a performance advantage [R2].

Deploy lightweight on-device inference only when the chosen device model, OS permissions and module compatibility have been tested; route heavier or updated models to a Python service. The core web experience can use React DOM where native-styled widgets harm semantics, accessibility or SEO. A Flutter alternative remains viable if the team has stronger Dart skills and requires pixel-identical UI; Kotlin Multiplatform becomes attractive if native platform capabilities dominate. SwiftUI is an iOS-specific companion option, not the only cross-platform FitFlow frontend [R3-R5].

Activity 2: Prepare Test Tasks and Success Criteria

2.1 Backend frameworks

Backend	Strengths	Trade-offs	FitFlow use
NestJS on Node.js / TypeScript	Modules, dependency injection, validation, WebSocket gateways; shares TypeScript skill set [R6].	Requires architectural discipline; CPU-heavy inference should not run in API event loop.	Proposed core API, workout, progress, nutrition metadata and social gateway.
Python / FastAPI	Python ML ecosystem, async endpoints and typed request models [R7].	Separate service and language add deployment/observability work.	Dedicated inference service, not default owner of accounts.
Go / HTTP frameworks	Compiled services, strong concurrency, lean binaries.	Another language and ML integration overhead for this team.	Potential later high-throughput gateway; unnecessary initial complexity.
Node.js / Express	Lightweight, flexible and case-study-aligned [S1].	Structure, guards and validation require more conventions than NestJS.	Low-complexity alternative; NestJS may run on Express adapter.

2.2 Database choices

Database	Scale and query profile	Health/privacy considerations	Real-time & cost considerations
Managed PostgreSQL	Relational transactions, joins, indexes and JSONB; row security policies available [R9,R10].	Good fit for user-owned plans, meals and audit records; configure RLS, encryption, backups, region.	Real-time requires separate event layer; managed operational and connection costs.
MongoDB Atlas	Flexible documents and indexing; cross-document design needs care.	Atlas offers BAA pathway where applicable, but customer duties remain [R14,R19].	Change streams possible; plan-based costs and provider dependence.
Cloud Firestore (Firebase)	Hierarchical documents, real-time listeners, offline client cache [R8].	Rules/permissions and product-specific BAA scope must be checked; not automatically HIPAA-compliant [R18].	Easy live sync; reads/writes/listeners can affect bills; complex cross-document analytics.
Amazon DynamoDB	Managed key-value/document scale with access-pattern-based modeling.	Use region, IAM, encryption and eligibility check for PHI [R11].	Scales well; joins/ad-hoc progress analytics need design or another store.

2.3 Identity and authorization options

Option	Strengths	Limitations / FitFlow decision
Amazon Cognito	Managed user pools, OAuth2/OIDC tokens, MFA options, AWS integration; HIPAA-BAA eligible per AWS docs [R12].	AWS complexity and pricing to assess; app still enforces owner/coach/group permissions. Proposed identity service.
Firebase Authentication	Convenient app sign-in, fits Firebase stack.	Separate per-product BAA eligibility review if PHI is in scope; does not replace API authorization.
Auth0	Flexible identity flows, enterprise integrations and policy tooling.	Contract/tier/BAA terms must be verified; vendor cost and integration work vary.
Supabase Auth	Integrated PostgreSQL-oriented developer workflow.	HIPAA PHI use requires signed BAA and enabled add-on on supported hosted setup [R13].

2.4 Combined technology choice

Layer	Selected component	Reason / limitation
Mobile + web	React Native / Expo + React web	Matches case-study mobile decision, shared TypeScript domain code [S1,R1].
API and realtime	NestJS / Node.js + Socket.IO	Structured services and scoped event delivery [R6].
AI	FastAPI + Python model service; optional on-device TFLite	Isolate CPU/ML inference; allow device-side privacy where feasible [S1,R7].
Authoritative data	Managed PostgreSQL	Transactional user-owned fitness, meal, progress and group records [R9,R10].
Cache	Managed Redis (optional at MVP)	Rate limiting and short-lived non-sensitive caches; no PHI by default.
Identity	Amazon Cognito + backend authorization	Managed tokens plus explicit server-side policies [R12].
Media	Private object storage with expiring URLs	Meal photos minimized, scanned, TTL-deleted.
Cloud	AWS managed services as proposed deployment	One cloud simplifies identity/data operations; verify service BAA scope before PHI [R11].

2.5 Combined technology choice

HIPAA does not automatically apply to every consumer fitness app: its applicability depends on the organization's role, data and US covered-entity/business-associate relationships. If PHI is in scope, execute a BAA and verify that EVERY service touching PHI is covered; vendor eligibility or a BAA alone does not make FitFlow compliant [R11,R14,R18,R19]. If GDPR applies, identify lawful bases and additional conditions for special-category health data, implement data-subject rights and document processing with qualified review. Jurisdictional requirements are to be confirmed before production; the case-study statement about GDPR/CCPA is a scenario claim, not proof of a deployed compliant service [S1].

- Security controls: TLS, managed encryption at rest, least-privilege service accounts, MFA for staff, secrets manager, audit logs without medical detail, periodic access reviews and patching.
- Authorization: verify Cognito tokens server-side; API roles user/trainer/moderator/admin; user-owned rows and explicit group membership checks; PostgreSQL RLS is defense-in-depth, not a substitute for application checks [R9,R12].

- AI and nutrition: collect minimal inputs, redact model logs, use short photo retention, show confidence/uncertainty and allow manual correction; no automated diagnosis or medical advice.
- Operational guardrails: backups and tested restore, rate limits, incident response, opt-in social sharing, moderation, data deletion/export, accessible offline UI.

Cost note: no fixed monthly vendor price is asserted. Compare total cost using expected monthly active users, API requests, inference GPU/CPU seconds, data size, photo storage, egress, support tiers and contractual compliance add-ons.

Activity 3: Technology comparison and weighted decision matrix

3.1 Method, weights and scoring conventions

A 1-5 scale expresses estimated FitFlow suitability, not measured vendor benchmarks: 1 = poor fit; 2 = below-average fit; 3 = workable with major trade-offs; 4 = strong fit; 5 = strong fit with fewer expected integration barriers. All candidates are assumed competently engineered; no stack alone guarantees security or legal compliance. A candidate score equals $\text{SUM}(\text{weight} \times \text{rating}) / 100$, giving a 1-5 weighted average.

Criterion	Weight	FitFlow basis
Security / privacy readiness	20%	Sensitive profiles + sharing
Runtime performance	15%	Responsive workouts and tracking
Development speed	13%	Startup time-to-market
Scalability	12%	Growth after retention recovery
AI / ML fit	12%	AI plans and food recognition
Real-time features	10%	Challenges, groups, alerts
Maintainability	10%	Mid-sized team ownership
Lifecycle cost	8%	Managed service + inference bills

Stack	Sec.	Perf.	Speed	Scale	AI	Live	Maint.	Cost	Score	Index
A - Proposed RN/Nest/PG/Cognito/AI	4	4	5	4	5	5	4	4	4.35/5	87.0/100
B - Flutter/FastAPI/Mongo/Auth0	3	5	4	4	4	3	3	3	3.67/5	73.4/100
C - Kotlin/Go/Dynamo/Cognito	4	5	2	5	3	4	3	3	3.71/5	74.2/100
D - Native/Express/Firestore	3	5	2	4	3	5	2	2	3.31/5	66.2/100

3.2 Worked calculation and sensitivity check

Example for candidate A: $(4 \times 20 + 4 \times 15 + 5 \times 13 + 4 \times 12 + 5 \times 12 + 5 \times 10 + 4 \times 10 + 4 \times 8) / 100 = 4.35/5$. This is a transparent assignment rubric, not a prediction of real-world speed, security or costs.

Sensitivity: doubling lifecycle-cost priority would change the decision weights; a different hiring profile could raise Flutter or Kotlin scores. Therefore the matrix should be re-scored with the actual engineering team after a short camera/ML/web spike, estimated cloud usage and compliance review. Candidate A is the proposed architecture for this coursework, not an objectively universal best stack.

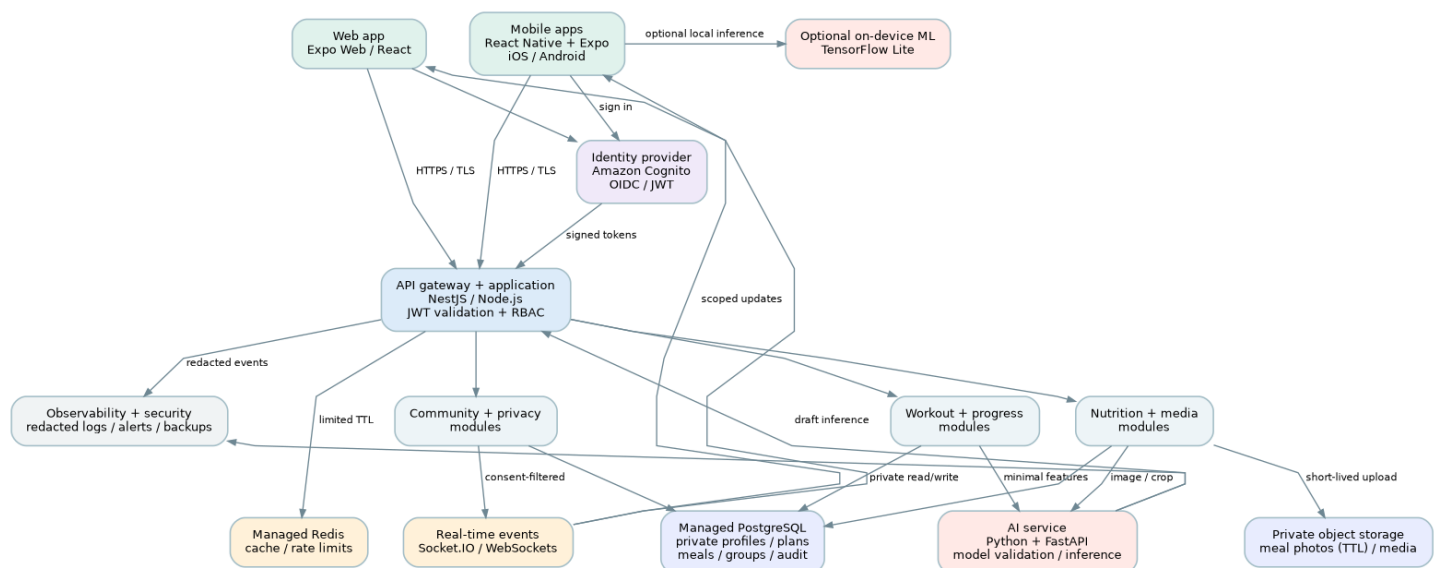
3.3 Decision rationale and case-study reconciliation

Candidate A aligns with the scenario's React Native and Node.js choice while replacing unstructured Express endpoints with NestJS conventions and replacing Firebase as the authoritative record store with managed PostgreSQL. The case study describes Firebase for real-time features, and that alternative remains valid when rapid synchronization is the overriding requirement [S1,R8]. For this proposal, database transactions and relational control over profiles, workout sessions, meals, trainer-client grants and per-group privacy justify PostgreSQL plus a separate WebSocket layer [R6,R9]. Cognito consolidates identity in the proposed AWS deployment but does not itself grant a trainer access to a user's records [R12].

Activity 4: High-level architecture

4.1 Proposed architecture diagram

Figure 1. Logical design for FitFlow; arrows show planned data/control paths, not deployed connections. Source: original student design informed by [S1, R1, R6-R12].



4.2 Component responsibilities

Component	Responsibility	Data boundary / scaling
Expo mobile + web	Home, AI Daily Flow, planner, progress, community, nutrition; client cache.	No service secrets; device-encrypted cached sensitive records.
Cognito / identity	Sign-in, MFA, token issuance, session expiry.	JWT checked by API; separate owner/team-role lookup.
NestJS application	Validate inputs, enforce authz, coordinate business modules, write audit trail.	Stateless workers, rate limits, autoscale based on demand.
Python FastAPI AI	Recommend workouts and infer food labels with explanations/confidence.	Minimum necessary features; inference job TTL and model version.
PostgreSQL	Source of truth for user, plan, session, meal, group and consent metadata.	Private network, backups, migration, scoped access.
Redis / WebSocket	Ephemeral cache, throttling, scoped live challenge events.	No durable health history in cache; expiration + limits.
Object store	Private temporary food photos and non-sensitive public assets.	Signed short-lived uploads; malware scan; lifecycle deletion.
Monitoring	Availability, errors, redacted audit and incident alerts.	No raw meal images/health details in logs.

4.3 Critical data flows

Flow A - Personalized workout planning

1. User selects goal, available minutes, equipment and optional constraints in Daily Flow. 2. Mobile/web obtains an identity token; NestJS validates it and confirms record ownership. 3. API retrieves only needed training history from PostgreSQL. 4. AI service receives a minimal feature set and model version, returns a candidate plan plus explanation/uncertainty. 5. API checks exercise safety rules and stores a draft. 6. User reviews, edits and approves before workout starts. 7. Completed sets and progress are saved transactionally; offline changes sync with conflict handling. No medical diagnosis is provided.

Flow B - Private social sharing

1. User joins a group or private challenge; NestJS verifies invitation and group membership. 2. Post creator chooses audience; server enforces visibility for every read/write. 3. Approved events publish an audience-filtered channel update via WebSockets. 4. Moderator actions can hide/report content. 5. Store only necessary engagement data and never publish private progress or health history by default.

Flow C - Nutrition logging

1. User chooses a meal photo or manual entry. 2. API issues an authenticated, short-lived upload link for private storage. 3. Scan the upload and strip unnecessary metadata before restricted inference. 4. FastAPI estimates possible food/portions and returns confidence; user corrects

labels and quantities. 5. Persist only approved meal nutrition in PostgreSQL. 6. Expire/delete source image under documented retention policy; support data export/deletion requests.

4.4 Cross-cutting quality and integration

Concern	Design control	Verification before production
Availability / scalability	Stateless API, managed DB, cache limits, backpressure, job timeout.	Load tests, failover, restore drill.
Privacy	Encryption, minimum data, per-user authorization, opt-in sharing, audit.	Threat model, privacy assessment, role tests.
AI safety	Human-editable results, exercise constraints, fallbacks, model monitoring.	Trainer review, bias tests, model performance on device.
Accessibility	Readable text, keyboard/screen-reader navigation, reduced motion.	Representative users and accessibility testing.
Offline consistency	Local drafts and idempotent sync operations.	Airplane-mode and conflict scenarios.
Integration	Versioned HTTPS API and schema-validated events.	Contract tests; OAuth/token expiry scenarios.

4.5 Architecture Decision Record (ADR-001)

ADR field	Record
Title	FitFlow mobile-first cross-platform stack with isolated AI and relational source of truth
Status / date	Proposed for coursework / 16 September 2026
Context	Declining retention, nutrition friction, personalization gap, private social needs and modest team size [S1].
Decision	React Native/Expo frontend; NestJS core API; Python FastAPI AI service; managed PostgreSQL; Cognito; WebSockets; optional Redis/on-device inference.
Alternatives	Flutter full stack; KMP shared logic; separate Swift/Kotlin; Firestore-first real-time app; MongoDB schema flexibility.
Reasons	Shared TS workflow, documented web path, modular real-time APIs, transactional records, isolated model workloads [R1,R6,R7,R9,R12].
Consequences	Extra AI service and WebSocket deployment; separate identity integration; operational overhead vs all-Firebase approach.
Risk controls	Spike web/camera integration, verify BAA eligibility, test per-group authz, budget ML, security review and model evaluation.
Revisit trigger	Poor web accessibility, ML bridge incompatibility, unsustainable ops cost, changed PHI obligations or failed load tests.

References and source provenance

Official documentation consulted 16 September 2026. Bracketed IDs in the report identify sources; vendor capabilities, eligibility and pricing can change. Course-provided sources [S1-S4] are not public URLs.

[S1] IT3060 FitFlow Redesign Case Study (course-provided PDF). CourseWeb case study; supplied in this conversation.

[S2] IT3060 Lab Exercise 05 (course-provided PDF). CourseWeb lab sheet; supplied in this conversation.

[S3] IT3060 Lab Exercise 01 research (student-provided PDF). Five coded participants P1-P5; student-provided report.

[S4] IT3060 Lab Exercise 04 usability report (student-provided PDF). P01-P10 records are labelled simulated in that report.

[R1] Expo: Develop websites with Expo. <https://docs.expo.dev/workflow/web/>

[R2] Expo: React Native New Architecture. <https://docs.expo.dev/guides/new-architecture/>

[R3] Flutter: Web support. <https://docs.flutter.dev/platform-integration/web>

[R4] Kotlin Multiplatform: Supported platforms.
<https://kotlinlang.org/docs/multiplatform/supported-platforms.html>

[R5] Apple Developer: SwiftUI apps.
<https://developer.apple.com/documentation/technologyoverviews/swiftui>

[R6] NestJS: WebSocket gateways. <https://docs.nestjs.com/websockets/gateways>

[R7] FastAPI: Concurrency and async / await. <https://fastapi.tiangolo.com/async/>

[R8] Firebase: Cloud Firestore capabilities. <https://firebase.google.com/docs/firestore>

[R9] PostgreSQL: Row Security Policies. <https://www.postgresql.org/docs/17/ddl-rowsecurity.html>

[R10] PostgreSQL: JSON data types and indexes. <https://www.postgresql.org/docs/16/datatype-json.html>

[R11] AWS: HIPAA compliance and eligible services.
<https://aws.amazon.com/compliance/hipaa-compliance/>

[R12] AWS: What is Amazon Cognito?
<https://docs.aws.amazon.com/cognito/latest/developerguide/what-is-amazon-cognito.html>

[R13] Supabase: HIPAA compliance and customer responsibilities.
<https://supabase.com/docs/guides/security/hipaa-compliance>

[R14] MongoDB Atlas: HIPAA compliance and shared responsibility.
<https://www.mongodb.com/docs/atlas/architecture/current/compliance/hipaa/>

[R15] GitHub: Creating a new repository. <https://docs.github.com/en/repositories/creating-and-managing-repositories/creating-a-new-repository>

[R16] GitHub: Managing protected branches.
<https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches>

[R17] GitHub CLI: gh repo create. https://cli.github.com/manual/gh_repo_create

[R18] Google Cloud: HIPAA compliance and BAA scope.
<https://cloud.google.com/security/compliance/hipaa-compliance>

[R19] MongoDB: HIPAA compliance and BAA does not ensure compliance.
<https://www.mongodb.com/products/platform/trust/hipaa>